

Meal-Planner Safety Harness

High-Level Design · An AI agent that *lives inside* a safety harness

The agent only **proposes** a meal plan. The harness alone decides what reaches the user. **All safety lives in the gap between *propose* and *apply*** — never inside the model.

1 · The real-world problem

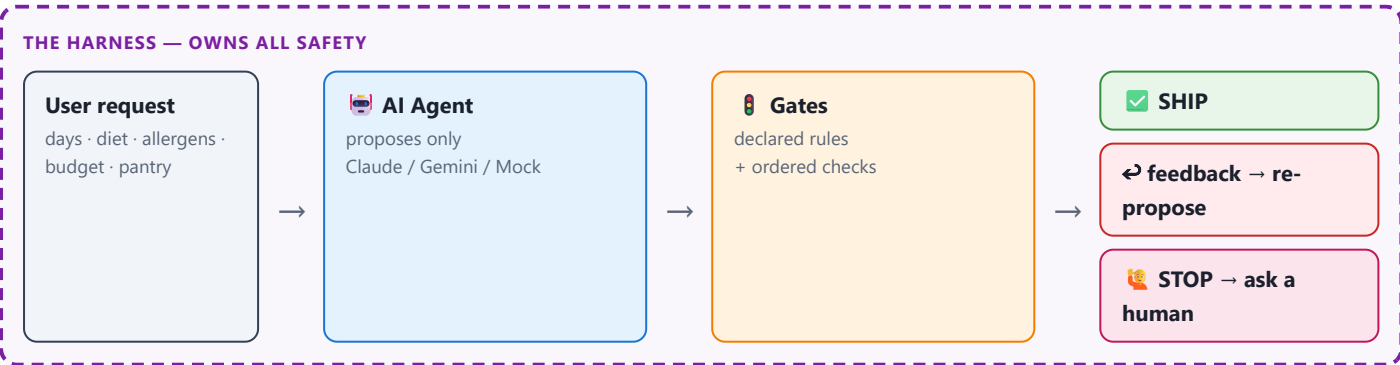
People let AI plan what they eat — but a meal plan is **acted on**, not a throwaway chat reply. A *fluent* AI is not a *safe* AI: the same model that writes a great plan will, just as confidently, write a dangerous one.

A confident AI can output...	...and a real person is harmed
Peanut butter for a peanut-allergic user	Anaphylaxis
A 700 kcal/day “weight-loss” plan	Starvation-level diet, medically unsafe
An ingredient nobody confirmed you own	A plan you literally cannot cook

You cannot trust the model to police itself. The component generating the plan must not be the final word on whether it is safe to serve.

2 · How we address it — *propose vs. apply*

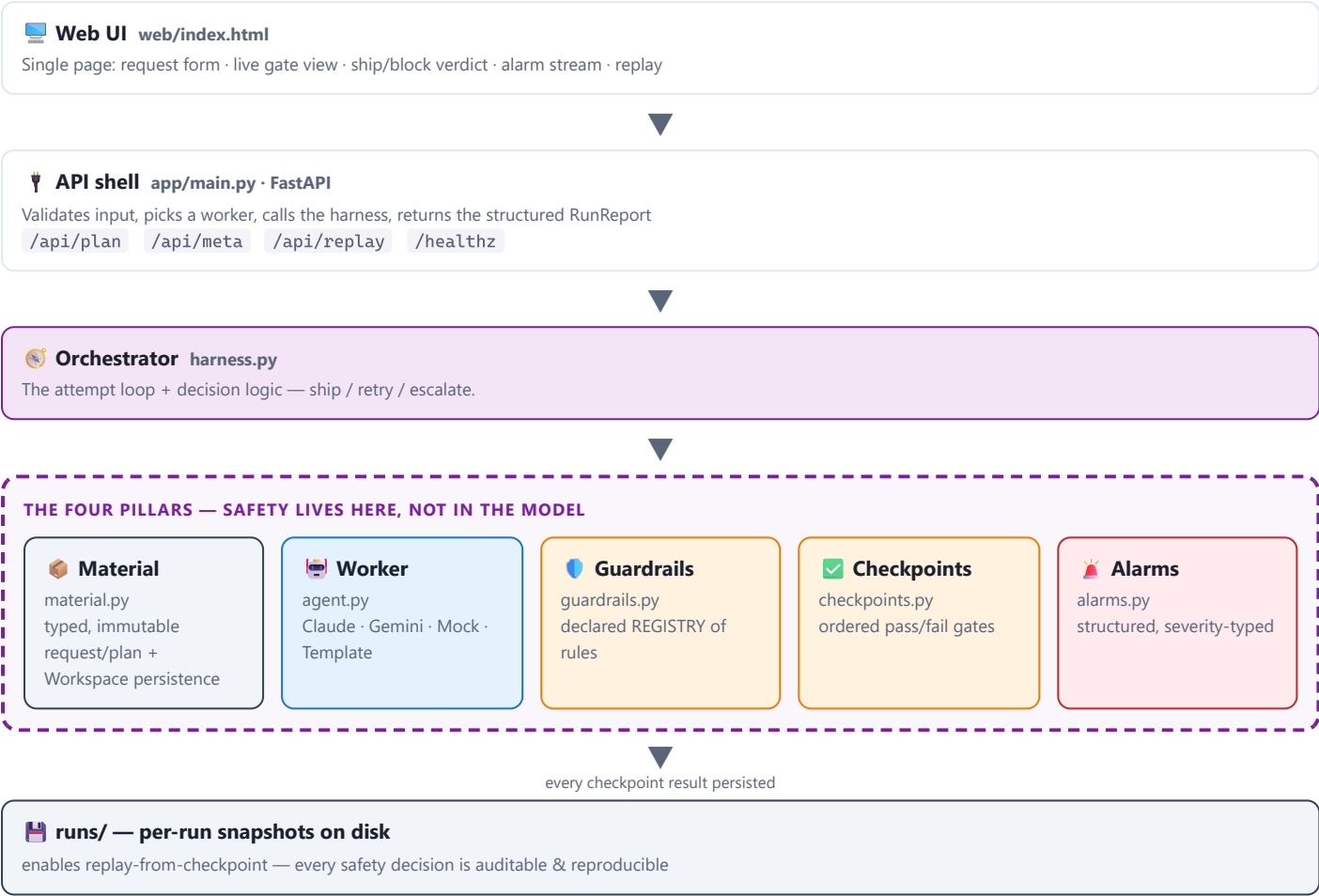
We don’t build a “safer model.” We wrap **any** model in a harness that owns every safety decision. The agent is **powerful but untrusted**: it makes proposals; nothing it says is authoritative until the harness clears it.



The agent is fully **swappable** because it speaks one contract: `propose(request, feedback) → plan`. Swap Claude for Gemini for a mock — the safety guarantees don’t move, because they were never in the model.

3 · System architecture

A thin web shell on top of the orchestrator, which sequences four independent pillars. **The orchestrator reimplements none of them — it only sequences.**



Data that flows through the harness

Object	What it carries
<code>PlanRequest</code> (in)	Normalized, immutable user need — the only thing a worker is handed.
<code>MealPlan</code> (out)	The worker's <i>proposal</i> — never authoritative until gated.
<code>CheckpointResult</code>	PASS/FAIL + criteria + evidence, snapshotted per stage.
<code>Alarm</code>	type · severity · context · recommended action · source.
<code>RunReport</code>	Final verdict + every checkpoint + every alarm + the plan (if shipped).

4 · The four pillars in one glance

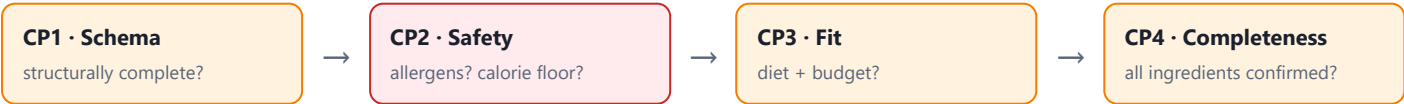
Pillar	Role	Key property
 Guardrails guardrails.py	<i>Declare</i> what “safe” means	Every rule is a data object in one <code>REGISTRY</code> list — no safety logic hidden in the model or orchestrator . Tuning safety = editing the list.
 Checkpoints checkpoints.py	<i>Gate</i> the proposal, in order	Explicit PASS/FAIL + the evidence judged on. SAFETY runs before anything can ship ; a CRITICAL failure stops the line.
 Material material.py	Move request <i>in</i> / proposal <i>out</i>	Typed & immutable. The agent never touches disk or raw input — that decoupling is exactly what makes it swappable.
 Alarms alarms.py	<i>Signal</i> what went wrong	Structured objects (type · severity · context · recommended action), streamed as JSONL — not log lines. CRITICAL alarms are blocking.

The declared guardrail registry — the entire safety surface, in one editable list

Guardrail	Severity	Rule
<code>ALLERGEN_EXCLUSION</code>	CRITICAL	No ingredient matching a user allergen (alias-expanded: “peanut” → groundnut, arachis...) may appear.
<code>CALORIE_FLOOR</code>	CRITICAL	No day, and no stated target, may fall below the 1200 kcal safety floor.
<code>CALORIE_CEILING</code>	WARNING	No day may exceed the 5000 kcal sanity ceiling.
<code>INGREDIENT_CONFIRMED</code>	WARNING	Every ingredient is pantry-stocked or explicitly flagged <code>buy: .</code>
<code>DIET_COMPLIANCE</code>	WARNING	All meals respect the declared diet type (vegan, pescatarian...).
<code>BUDGET_LIMIT</code>	WARNING	Total plan cost stays within the weekly budget.

A failed guardrail returns structured `violations`, a worker-readable `feedback` string, and an `auto_recoverable` flag — the line between “retry the worker” and “stop and ask a human.”

The ordered checkpoint pipeline — order is itself a safety property



CP2 SAFETY runs before any stage that could ship a plan; a CRITICAL failure there **stops the line** so no downstream gate ever evaluates an unsafe plan.

5 · How the agent works & communicates

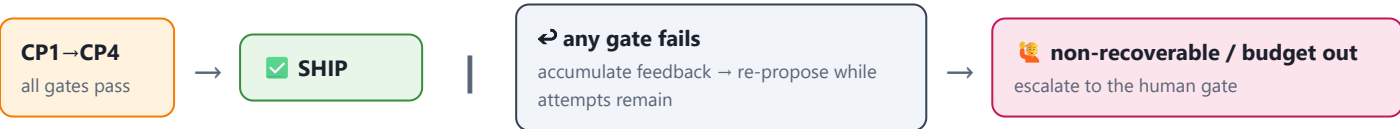
The agent never gets the last word, and never works blind. Each failed gate hands back a **structured, worker-readable feedback string**. Feedback is **cumulative** — the harness remembers every correction, so the agent *converges* instead of trading one defect for another.

Attempt 1	Harness → Agent: <code>propose(request, feedback="")</code> Agent → Harness: plan with peanut butter + an unflagged ingredient	CP2 SAFETY — allergen → line STOPS later gates never run on an unsafe plan
Attempt 2	Harness → Agent: <code>propose(request, "remove peanut...")</code> Agent → Harness: peanut fixed, ingredient still unflagged	CP4 — unconfirmed ingredient
Attempt 3	Harness → Agent: <code>propose(request, "remove peanut... + flag buy:")</code> Agent → Harness: both defects fixed	ALL GATES PASS ✓ → SHIP
Non-recoverable	e.g. a user's stated 700 kcal target — Harness → Human: <i>EscalationPacket</i> "approve override or reject?"	Human rejects → BLOCK never auto-guesses a number

Core guarantee: the agent's behavior changes in response to the harness — not the other way around.

6 · How the final outcome is produced

Every run ends in exactly one verdict. Safety gates **stop the line**: a CRITICAL failure means no later gate runs, so an unsafe plan can never be "saved" by a downstream check.



Outcome	Meaning
SHIPPED	Cleared every gate (often after self-correction).
BLOCKED · pending human	Non-recoverable safety issue, no human wired in → safe default: halt.
BLOCKED · rejected	A human reviewed and declined the override.
BLOCKED · loop exhausted	Agent couldn't satisfy constraints in budget → escalates; never ships a guess.

Every checkpoint result is **persisted to disk**, so any run can be **replayed from any checkpoint forward without re-invoking the agent**.

7 · Technology — how it's built

Layer	Stack
Frontend	Single static HTML page + vanilla JS (zero build)
Backend	Python 3.12 · FastAPI · Uvicorn · Pydantic
Safety core	Pure Python, zero third-party deps — runs & is testable with no API key
Live agents	Claude <code>sonnet-4-6</code> · Gemini <code>2.5-flash</code> (lazy-imported)
Offline agents	Mock + Template — deterministic, demo without keys
Deploy	GitHub → Railway (Nixpacks · Procfile · env-var keys)

Why this shape

- SDKs are imported **lazily** → the whole safety engine runs offline; judges see it work with no key.
- Workers are **drop-in**: the same harness drives Claude, Gemini, or a mock with **zero harness changes**.
- API keys live only in the server environment — never in code or git.
- The web layer is a **thin shell**: validate → pick worker → `Harness.run` → report. **All safety stays in the harness.**

8 · Hackathon requirements — and how this project meets each

Class	Requirement	How we met it	Where
MUST	Four pillars implemented & separate from the worker	Each pillar a distinct module; worker isolated behind a protocol; orchestrator only sequences. ✓	guardrails · checkpoints · material · alarms · agent
MUST	Agent behavior changes meaningfully on feedback	Cumulative feedback loop: agent fixes allergen, then unflagged item, converging over 3 attempts. ✓	harness.py loop · §5
MUST	Guardrails declared ; checkpoints explicit pass/fail	Declared <code>REGISTRY</code> ; ordered <code>PIPELINE</code> , each stating criteria + PASS/FAIL + evidence. ✓	REGISTRY · PIPELINE
MUST	Alarms produce structured output	Every alarm: type · severity · context · recommended action · source; 9 named types; JSONL stream. ✓	alarms.Alarm · AlarmType
MUST	Runs on a real input at demo time	Live web app on Railway + real request fixtures (pescatarian, peanut/shellfish allergy). ✓	inputs/*.json · live URL
MUST	A HARNESS.md on architecture & design	Full HARNESS.md + this HLD as the higher-altitude companion. ✓	HARNESS.md · ARCHITECTURE.md
SHOULD	Swappable agent — no harness changes	The <code>Worker</code> protocol (<code>propose</code>) is the only contract; 4 workers drop in unchanged. ✓	agent.Worker
SHOULD	Checkpoints persisted ; replay forward	<code>Workspace</code> snapshots each stage; <code>replay_from()</code> re-runs the rest without calling the worker. ✓	Workspace · replay_from
SHOULD	Human-in-the-loop escalation	Non-recoverable failures build an <code>EscalationPacket</code> → <code>human_gate</code> ; default = halt. ✓	EscalationPacket · human_gate
BONUS	Second worker swapped in at demo	Template & Gemini workers swap into the same pipeline live in the UI. ✓	Template/Gemini Worker

Coverage: all 6 **MUST** · all 3 **SHOULD** · the **BONUS** — met and demonstrable in the live app.

The one-line pitch

Don't make the model safe — put the model in a harness that's safe. The agent proposes; declared guardrails, ordered checkpoints, structured alarms, and a stop-and-ask-a-human gate decide. Swap the model freely; the guarantees never move.